PES UNIVERSITY

CELEBRATING 50 YEARS

Data Structures

Dilip Kumar Maripuri

Computer Applications

PES UNIVERSITY

Data Structures

Session : Applications of Graphs

Dilip Kumar Maripuri
Computer Applications

Data Structures
Applications of Graphs

PES UNIVERSITY

- **Find if a path of length n exists between a pair of vertices**
  - Matrix Exponentiation of Adjacency Matrix
  - Modified DFS

Data Structures

Matrix Exponentiation Method

PES UNIVERSITY

- **Understanding the Problem**
  - Given a directed or undirected graph represented as an **Adjacency Matrix**, we want to determine whether a **path of exactly length** *n* exists between two vertices.
  - This problem can be efficiently solved using **Matrix Exponentiation**, a technique that allows us to compute the **n-th power of a matrix efficiently using logarithmic exponentiation**.

Data Structures
Matrix Exponentiation Method

PES UNIVERSITY

- **Adjacency Matrix Representation**
- The adjacency matrix A of a graph with V vertices is a $V \times V$ matrix, where:
  - $A[i][j] = 1$ if there is a **direct edge** from vertex i to vertex j.
  - $A[i][j] = 0$ otherwise.
- For a **weighted graph**:
  - $A[i][j]$ stores the **weight** of the edge between i and j (or infinity if no edge exists).

Data Structures
Matrix Exponentiation Method

PES UNIVERSITY

- **Key Property of Matrix Exponentiation**
- If we compute A^n (raising A to the power n), then:

$$(A^n)[i][j] \tag{1}$$

gives us the **number of distinct paths of length n from vertex i to vertex j.**

Thus, **if** $(A^n)[i][j] > 0$, a path of length n exists between i and j.

Data Structures
Matrix Exponentiation Method

- **Matrix Multiplication and Its Significance**
- To understand how matrix exponentiation helps, let look at **matrix multiplication**:
- For two matrices A and B, the product $C = A \times B$ is defined as:

$$C[i][j] = \sum_{k=0}^{V-1} A[i][k] \times B[k][j] \tag{2}$$

Data Structures
Matrix Exponentiation Method

PES UNIVERSITY

- **Interpreting Matrix Multiplication in Graphs**
  - If $A[i][k] = 1$ and $B[k][j] = 1$, then there exists a **path of length 2** from i to j through k.
  - Repeating this process finds paths of increasing length.
- By computing A^n, we find paths of length n in an efficient manner.

Data Structures
Matrix Exponentiation Method

PES UNIVERSITY

- **Matrix Exponentiation Using Fast Powering**
- Instead of computing A^n by multiplying A repeatedly n times (which takes $O(V^3 n)$ time), we use **fast exponentiation** (or exponentiation by squaring), which reduces this to $O(V^3 \log n)$.
- **Exponentiation by Squaring**
  - If n is **even**: $A^n = (A^{n/2}) \times (A^{n/2})$
  - If n is **odd**: $A^n = A \times (A^{n-1})$
  - Repeat until n becomes 1.

Data Structures
Matrix Exponentiation Method

- Consider the adjacency matrix for a directed graph with 4 vertices:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix} \tag{3}$$

- **Finding Paths of Length** $n = 2$ Compute $A^2 = A \times A$:

$$A^2 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \tag{4}$$

Since $(A^2)[0][2] = 1$, there is a **path of length 2** from vertex 0 to 2.

Data Structures
Matrix Exponentiation Method

PES UNIVERSITY

- **Finding Paths of Length $n = 3$:** Compute $A^3 = A^2 \times A$:

$$A^3 = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \tag{5}$$

Since $(A^3)[0][3] = 1$, there is a **path of length 3** from vertex 0 to 3.

- **Interpretation:**
  - $(A^3)[0][3] = 1 \rightarrow$ There is a **path of length 3** from vertex 0 to 3.
  - $(A^3)[1][0] = 1 \rightarrow$ There is a **path of length 3** from vertex 1 to 0.
  - $(A^3)[2][1] = 1 \rightarrow$ There is a **path of length 3** from vertex 2 to 1.
  - $(A^3)[3][2] = 1 \rightarrow$ There is a **path of length 3** from vertex 3 to 2.

Data Structures

Matrix Exponentiation Method

PES UNIVERSITY

- **Problem** : We are given an adjacency matrix A, representing a directed graph:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

- We need to compute A^5, which tells us the number of paths of length 5 between any two vertices.

Data Structures

Matrix Exponentiation Method

PES UNIVERSITY

- We use **Fast Exponentiation** to compute A^5:
  1. Start with the base A.
  2. Use **exponentiation by squaring** to compute A^5:
     - Compute $A^2 = A \times A$.
     - Compute $A^4 = A^2 \times A^2$.
     - Compute $A^5 = A^4 \times A$.

Data Structures

Matrix Exponentiation Method

PES UNIVERSITY

- **Step 1: Compute** A^2 Matrix multiplication $A^2 = A \times A$:

$$A^2 = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Performing the multiplication:

$$A^2 = \begin{bmatrix} 1 & 0 & 1 & 2 \\ 1 & 0 & 0 & 1 \\ 0 & 2 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Matrix Exponentiation Method

- **Step 2: Compute** A^4 To compute $A^4 = A^2 \times A^2$, use A^2 from Step 1:

$$A^4 = \begin{bmatrix} 1 & 0 & 1 & 2 \\ 1 & 0 & 0 & 1 \\ 0 & 2 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & 1 & 2 \\ 1 & 0 & 0 & 1 \\ 0 & 2 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Performing the multiplication:

$$A^4 = \begin{bmatrix} 1 & 1 & 2 & 4 \\ 1 & 0 & 2 & 2 \\ 0 & 3 & 1 & 3 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Matrix Exponentiation Method

- **Step 3: Compute** A^5 To compute $A^5 = A^4 \times A$, use A^4 from Step 2 and the base matrix A:

$$A^5 = \begin{bmatrix} 1 & 1 & 2 & 4 \\ 1 & 0 & 2 & 2 \\ 0 & 3 & 1 & 3 \\ 0 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Performing the multiplication:

$$A^5 = \begin{bmatrix} 2 & 1 & 3 & 5 \\ 1 & 1 & 2 & 4 \\ 1 & 1 & 2 & 2 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

- **Interpretation of A^5** Each entry $(A^5)[i][j]$ gives the number of paths of length 5 from vertex i to vertex j. For example:
  - $(A^5)[0][3] = 5$: There are 5 paths of length 5 from vertex 0 to vertex 3.
  - $(A^5)[1][2] = 2$: There are 2 paths of length 5 from vertex 1 to vertex 2.
  - $(A^5)[3][j] = 0$: No paths of length 5 start from vertex 3 because it has no outgoing edges.

Data Structures

Matrix Exponentiation Method

PES UNIVERSITY

- Using **fast exponentiation**, we computed A^5 efficiently in three steps:
  1. Compute $A^2 = A \times A$.
  2. Compute $A^4 = A^2 \times A^2$.
  3. Compute $A^5 = A^4 \times A$.

  This approach avoids repeated multiplications and reduces complexity to $O(V^3 \log n)$.

Data Structures

Modified DFS Method

PES UNIVERSITY

- To determine if a path of exactly n edges exists between two vertices in a graph (directed or undirected), we can use a **modified Depth-First Search (DFS)**.

  Key Idea:

  - Perform a DFS traversal starting from the **source vertex**.
  - Track the **current path length** during the traversal.
  - Stop the traversal for a branch if:
    - The path length exceeds n.
    - The destination vertex is reached exactly when the path length is n.

Modified DFS Method

- **Steps for Modified DFS**
  1. **Track Path Length**: Maintain a counter to track the current path length during DFS.
  2. **Terminate Early**:
     - If the path length exceeds n, stop exploring further.
     - If the path length equals n, check if the destination vertex is reached.
  3. **Avoid Revisiting Vertices**: Use a `visited` array to prevent cycles in **undirected graphs**. This ensures each vertex is visited only once along a given path.

Data Structures

Modified DFS Method

- **Algorithm: Undirected Graph**
  1. **Input**:
     - G: The graph as an adjacency list.
     - u: Source vertex.
     - v: Destination vertex.
     - n: Desired path length.
  2. **Modified DFS**:
     - Start the DFS from vertex u.
     - Stop exploring further if the current path length exceeds n.
     - If the path length equals n, check if the current vertex is v.
  3. **Termination Condition**:
     - Return `True` as soon as the destination vertex is reached with path length n.
     - Continue exploring otherwise.

Data Structures

Transitive Closure of a Graph

PES UNIVERSITY

- The **transitive closure matrix** of a graph is derived by:
  1. **Matrix Multiplication:** Calculating successive powers of the adjacency matrix A ($A^2, A^3, A^4, \ldots$).
  2. **Logical OR Operations:** Combining each resulting matrix with the previous matrices using logical OR operations.

Data Structures

Transitive Closure of a Graph

PES UNIVERSITY

1. **Initialization:**
   - Start with the adjacency matrix A of the graph.
   - Let T (the transitive closure matrix) initially equal A.
2. **Iterative Updates:**
   - Compute successive powers of A, such as $A^2, A^3, \ldots$.
   - Combine the results using logical OR operations: $T = T \vee A^k$
   - Continue until T stops changing (i.e., no new paths are added).
3. **Termination:**
   - The final matrix T represents the transitive closure, where $T[i][j] = 1$ if there is any path (direct or indirect) from vertex i to vertex j.

Data Structures
Transitive Closure of a Graph

PES UNIVERSITY

- **Initial Matrix** A**:** The adjacency matrix A of the graph is:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

- **Step 1: Compute** A^2**:** Matrix multiplication $A^2 = A \times A$:

$$A^2 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Transitive Closure of a Graph

- **Step 2: Combine Using Logical OR**
- Combine A and A^2 using logical OR:

$$T = A \vee A^2 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \vee \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

- **Step 3: Compute A^3:**
- Matrix multiplication $A^3 = A^2 \times A$:

$$A^3 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Transitive Closure of a Graph

- **Step 4: Combine Again Using Logical OR**
- Combine T and A^3:

$$T = T \vee A^3 = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \vee \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

- **Step 5: Compute A^4:**

$$A^4 = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Transitive Closure of a Graph

- **Step 6: Combine Again Using Logical OR**

$$T = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \vee \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

- At this stage, T does not change. We have reached **convergence**.

Data Structures

OR Method

- For matrices A and B of the same size:
  - The logical OR operation between A and B can be expressed as:

$$C[i][j] = A[i][j] \vee B[i][j]$$

where:

- $C[i][j] = 1$ if $A[i][j] = 1$ or $B[i][j] = 1$.
- $C[i][j] = 0$ otherwise.

- This operation can be applied element-wise, which is straightforward and efficient.

OR Method

- The logical OR operation can also be achieved indirectly using **matrix addition**, followed by conversion:
  - Add A and B:

$$S = A + B$$

    where:
    - $S[i][j] = 0$ if $A[i][j] = 0$ and $B[i][j] = 0$.
    - $S[i][j] > 0$ if $A[i][j] = 1$ or $B[i][j] = 1$.
  - Convert the sum matrix S into a binary matrix C by replacing any positive values in S with 1:

$$C[i][j] = \begin{cases} 1 & \text{if } S[i][j] > 0, \\ 0 & \text{else.} \end{cases}$$

Data Structures

OR Method

PES UNIVERSITY

- **Given Matrices** Let:

$$A = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

- **Step 1: Add** A **and** B We add the corresponding elements of A and B:

$$S = A + B = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 1 & 0 \\ 1 & 1 & 2 & 0 \\ 0 & 1 & 1 & 2 \\ 2 & 0 & 1 & 1 \end{bmatrix}$$

Data Structures

OR Method

- **Step 2: Convert S to Binary** : Convert S to a binary matrix C by replacing all positive values in S with 1:

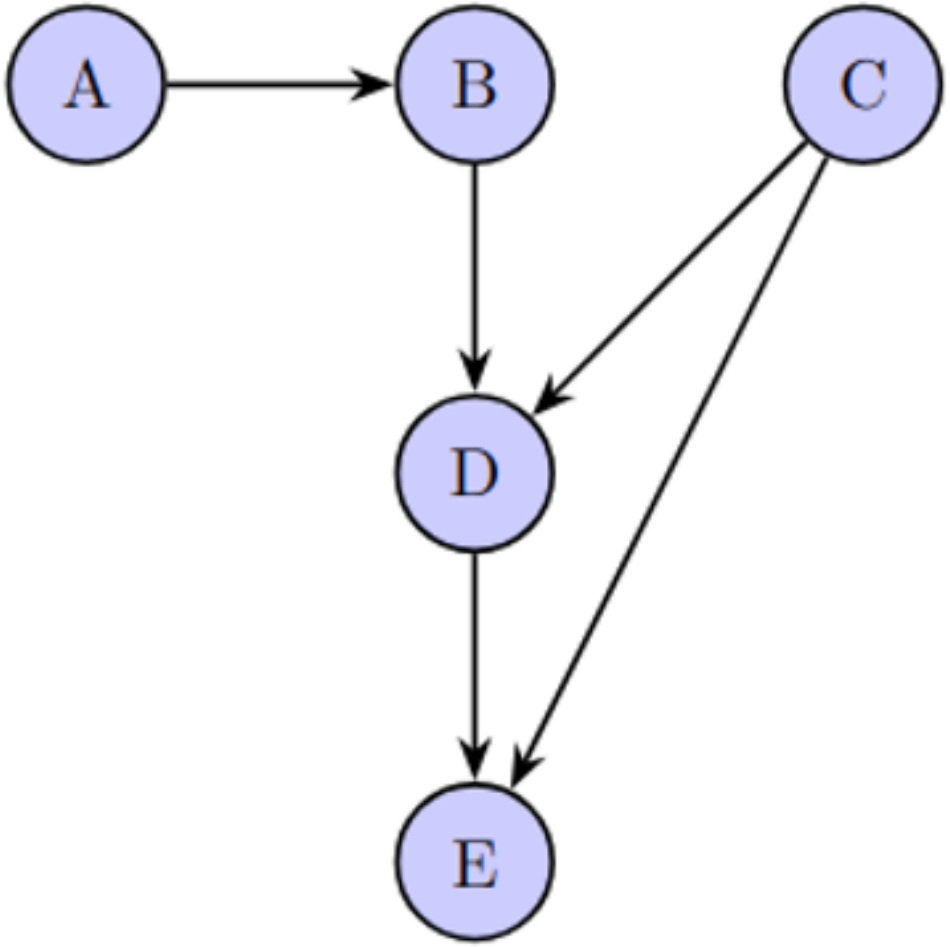
$$C = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

- **Final Result** : The result of the logical OR operation is:

$$C = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

Data Structures

Transitive Closure of a Graph



Transitive Closure of a Graph

- **Step 1: Adjacency Matrix Representation**

  From the given graph, the adjacency matrix A is derived as follows:

  - $A \rightarrow B$
  - $B \rightarrow D$
  - $C \rightarrow D, C \rightarrow E$
  - $D \rightarrow E$

  The adjacency matrix A is:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

- **Step 2: Compute A^2 (Paths of Length 2)** : We compute $A^2 = A \times A$:

$$A^2 = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Transitive Closure of a Graph

- **Step 3(a): Compute A^3 (Paths of Length 3)** :Now, compute $A^3 = A^2 \times A$:

$$A^3 = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

- **Step 3(b): Compute A^4 (Paths of Length 4)** :Now, compute $A^4 = A^2 \times A^2$:

$$A^3 = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Transitive Closure of a Graph

- **Step 4: Transitive Closure** : The transitive closure T is computed by taking the logical OR of $A, A^2, A^3, \ldots$, or until T stops changing.

  Step 4.1: Update T

$$T = T \vee A^2 = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \vee \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Transitive Closure of a Graph

- Step 4.2: Update T

$$T = T \vee A^3 = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \vee \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Transitive Closure of a Graph

- Step 4.3: Update T

$$T = T \vee A^4 = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \vee \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Transitive Closure of a Graph

- **Final Transitive Closure Matrix** The final transitive closure matrix is:

$$T = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Warshall's Algorithm

- Warshall's algorithm is a systematic method used to compute the **transitive closure** of a directed graph.
  1. **Transitive Closure:**
     - A directed graph's transitive closure determines whether there exists a path between any pair of vertices.
     - The resulting matrix R (reachability matrix) will have:

$$R[i][j] = \begin{cases} 1 & \text{if there is a path from } i \text{ to } j, \\ 0 & \text{otherwise.} \end{cases}$$

Data Structures
Warshall's Algorithm

- **Warshall's Algorithm:**
  - **Input:** Adjacency matrix A of size $n \times n$ (where n is the number of vertices).
  - **Output:** The transitive closure matrix R.

Data Structures
Warshall's Algorithm

PES UNIVERSITY

- **Steps of the Algorithm**
  1. **Initialization:**
     - Start with the adjacency matrix $R^{(0)} = A$.
     - The adjacency matrix indicates direct edges between vertices.
  2. **Iteration Over Intermediate Vertices:**
     - For each vertex k, check all pairs of vertices (i, j) to see if a path exists through k.
     - Update the reachability matrix:

$$R^{(k)}[i][j] = R^{(k-1)}[i][j] \vee \left(R^{(k-1)}[i][k] \wedge R^{(k-1)}[k][j]\right)$$

     - If i can reach j through k, set $R[i][j] = 1$.
  3. **Repeat for All Vertices:**
     - Perform the above operation for $k = 1, 2, \ldots, n$.
     - The matrix $R^{(n)}$ will represent the final transitive closure.

Data Structures
Warshall's Algorithm

PES UNIVERSITY

- Consider a directed graph with 4 vertices (a, b, c, d):
- **Initial Adjacency Matrix:**

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

- **Step-by-Step Execution**
- **Initialization:**
  - Start with $R^{(0)} = A$:

$$R^{(0)} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

Data Structures
Warshall's Algorithm

- **Iteration for $k = 1$:**
  - Check paths through vertex 1.
  - Update $R^{(1)}$:

$$R^{(1)} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

- **Iteration for $k = 2$:**
  - Check paths through vertex 2.
  - Update $R^{(2)}$:

$$R^{(2)} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

Warshall's Algorithm

- **Iteration for $k = 3$:**
  - Check paths through vertex 3.
  - Update $R^{(3)}$:

$$R^{(3)} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

- **Iteration for $k = 4$:**
  - Check paths through vertex 4.
  - Update $R^{(4)}$:

$$R^{(4)} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Data Structures

Warshall's Algorithm

PES UNIVERSITY

$R^{(0)} =$

	a	b	c	d
a	0	1	0	0
b	0	0	0	1
c	0	0	0	0
d	1	0	1	0

$R^{(1)} =$

	a	b	c	d
a	0	1	0	0
b	0	0	0	1
c	0	0	0	0
d	1	**1**	1	0

$R^{(2)} =$

	a	b	c	d
a	0	1	0	**1**
b	0	0	0	1
c	0	0	0	0
d	1	1	1	**1**

$R^{(3)} =$

	a	b	c	d
a	0	1	0	1
b	0	0	0	1
c	0	0	0	0
d	1	1	1	1

Warshall's Algorithm

- **Final Transitive Closure:**

$$R = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

- **Interpretation:**
  1. $R[i][j] = 1$ indicates there is a path from vertex i to vertex j.
  2. The transitive closure matrix ensures all reachable vertices are connected.

Data Structures

Warshall's Algorithm

ALGORITHM *Warshall*$(A[1..n, 1..n])$

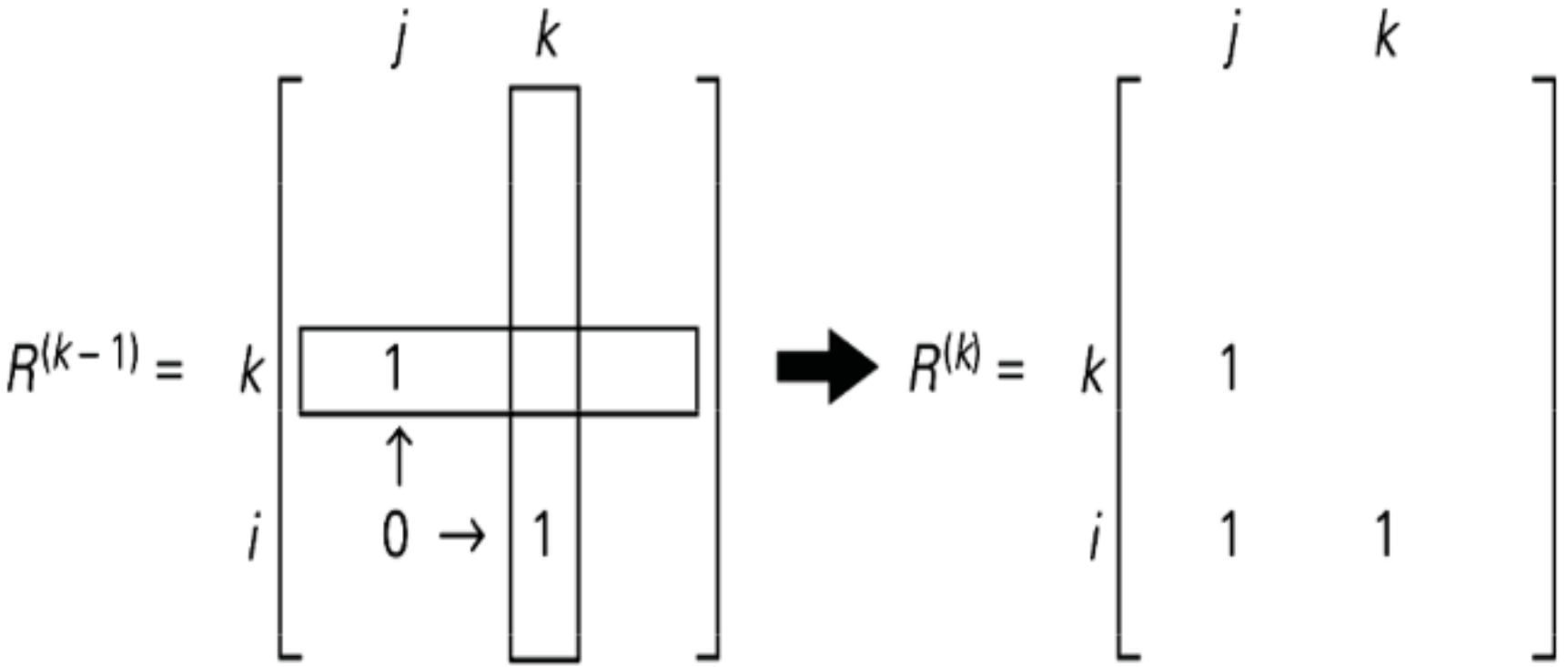
//Implements Warshall's algorithm for computing the transitive closure
//Input: The adjacency matrix A of a digraph with n vertices
//Output: The transitive closure of the digraph

$R^{(0)} \leftarrow A$

for $k \leftarrow 1$ **to** n **do**
  for $i \leftarrow 1$ **to** n **do**
    for $j \leftarrow 1$ **to** n **do**
      $R^{(k)}[i, j] \leftarrow R^{(k-1)}[i, j]$ **or** $(R^{(k-1)}[i, k]$ **and** $R^{(k-1)}[k, j])$

return $R^{(n)}$

Warshall's Algorithm



Thank You

Dilip Kumar Maripuri
Associate Professor
Department of Computer Applications
dilip.maripuri@pes.edu
8073212026